```python
import numpy as np


def int_to_bits_lsb(value: int, length: int) -> np.ndarray:

    if value < 0:
        raise ValueError("value >= 0 must be")
    if value >= (1 << length):
        raise ValueError("value size not combinian into import length")

    return np.array([(value >> i) & 1 for i in range(length)],
                    dtype=np.uint8)


def bits_to_int_lsb(bits):
    value = 0

    for i, bit in enumerate(bits):
        value |= int(bit) << i

    return value



class QAM_mapper:

    def __init__(self, M, bit_rate, E_symb_avg=1):

        self._M                     = M
        self._bit_rate              = bit_rate
        self._num_bits_in_qam       = int(np.log2(M))
        self._symbol_rate           = self._bit_rate/self._num_bits_in_qam
        self._T_symb                = 1/self._symbol_rate
        self._E_symb_avg            = E_symb_avg
        self._E_bit_avg             = E_symb_avg/np.log2(M)
        self._mod_signals           = self._get_mod_arr()



    def get_QAM4_signals_arr(self):
        arr_signals = np.zeros(4, dtype=complex)

        for numb in range(0, 4):
            bits = int_to_bits_lsb(numb, 2)
            arr_signals[numb] = (1/np.sqrt(2))*((1 - 2*bits[0]) + 1j*(1 - 2*bits[1]))

        return arr_signals * np.sqrt(self._E_symb_avg)


    def get_QAM16_signals_arr(self):
        arr_signals = np.zeros(16, dtype=complex)

        for numb in range(0, 16):
            bits = int_to_bits_lsb(numb, 4)
            arr_signals[numb] = (1/np.sqrt(10))*((1 - 2*bits[0])*(2 - (1 - 2*bits[2])) + 1j*(1 - 2*bits[1])*(2 - (1 - 2*bits[3])))

        return arr_signals * np.sqrt(self._E_symb_avg)


    def get_QAM64_signals_arr(self):
        arr_signals = np.zeros(64, dtype=complex)

        for numb in range(0, 64):
            bits = int_to_bits_lsb(numb, 6)
            arr_signals[numb] = (1/np.sqrt(42))*(  (1 - 2*bits[0])*(4 -(1 - 2*bits[2])*(2 - (1 - 2*bits[4]))) + 1j*(1 - 2*bits[1])*(4 -(1 - 2*bits[3])*(2 - (1 - 2*bits

        return arr_signals * np.sqrt(self._E_symb_avg)


    def get_QAM256_signals_arr(self):
        arr_signals = np.zeros(256, dtype=complex)

        for numb in range(0, 256):
            bits = int_to_bits_lsb(numb, 8)
            arr_signals[numb] = (1/np.sqrt(170))*(  (1 - 2*bits[0])*( 8 -(1 - 2*bits[2])*( 4 -(1 - 2*bits[4])*(2 - (1 - 2*bits[6])) ) )  +  1j*(1 - 2*bits[1]) * ( 8 -


        return arr_signals * np.sqrt(self._E_symb_avg)


    def _get_mod_arr(self):

        match self._M:
            case 4:
                return self.get_QAM4_signals_arr()
            case 16:
                return self.get_QAM16_signals_arr()
            case 64:
                return self.get_QAM64_signals_arr()
            case 256:
                return self.get_QAM256_signals_arr()
            case _:
                raise ValueError("There aren't modulation M=" + str(self._M))

        return 0


    def get_average_energy(self):

        return sum(x * x for x in self._mod_signals) / self._M


    def get_baseband_signal(self, bit_sequence):

        if ((np.size(bit_sequence) % self._num_bits_in_qam) != 0): raise ValueError("the number of bits is not a multiple of the number of bits per symbo")

        bit_seq_sz = np.size(bit_sequence)

        def baseband_signal_t(t):

            n = t // self._T_symb
            start_bits = int(n*self._num_bits_in_qam)
            end_bits = int((n+1)*self._num_bits_in_qam)

            if start_bits >= bit_seq_sz or end_bits >= bit_seq_sz: raise ValueError("There aren't bits for do modulating")

            cur_num = bits_to_int_lsb(bit_sequence[start_bits:end_bits])

            return self._mod_signals[cur_num]

        def baseband_signal(t_arr):
            if np.isscalar(t_arr):
                return baseband_signal_t(t_arr)
```

```
        result = np.zeros_like(t_arr, dtype=complex)

        for i in range(0, np.size(t_arr)):
            result[i] = baseband_signal_t(t_arr[i])

        return result

    return baseband_signal


def get_modulate_sequence_and_time(self, t_start, t_end, numb_of_points, bit_sequence):

    if (t_end - t_start) <= 0:  raise ValueError("t_end <= t_start")
    if ((t_end - t_start) // self._T_symb + 1) * self._num_bits_in_qam > np.size(bit_sequence): raise ValueError("PËnsufficient bits to form the baseband signal")
    if numb_of_points <= 0: raise ValueError("numb_points must be > 0")

    t = np.linspace(t_start, t_end, numb_of_points)
    result = np.zeros(numb_of_points, dtype=complex)

    for i in range(0, numb_of_points):
        n = (t[i] - t[0]) // self._T_symb

        start_bits = int(n*self._num_bits_in_qam)
        end_bits = int((n+1)*self._num_bits_in_qam)

        cur_signal_val = bits_to_int_lsb(bit_sequence[start_bits:end_bits])

        result[i] = self._mod_signals[cur_signal_val]

    return result, t
```